1. Software Design Specification

DICTIONARY_STDLIB

Document Control	
Author(s)	Michael Gardner
Version	0.1.0
Status	Draft
Status Date	2026-04-07
SPDX-License-Identifier	BSD-3-Clause
License File	See the LICENSE file in the project root
Copyright	© 2026 Michael Gardner, A Bit of Help, Inc.

Project Profile	
Variant	application
Library Role	N/A
Application Role	service
Assurance	non-spark
Execution	concurrent
Parallelism	bounded
Platform	server
Execution Environment	linux
Processor Architecture	amd64
Deployment	containerized

Table of Contents

1. Software Design Specification 1
2. Introduction 1
   - 2.1. Purpose 1
   - 2.2. Scope 1
   - 2.3. References 1
3. Architectural Overview 1
   - 3.1. Architecture Style 1
   - 3.2. Dependency Flow 1
4. Package Structure 2
   - 4.1. Directory Layout 2
   - 4.2. Package Descriptions 2
5. Type Definitions 2
   - 5.1. Generic Bounded_Text 2
   - 5.2. Instantiations 3
   - 5.3. Entry_Result (Lightweight Option) 3
   - 5.4. Store_Status (Operation Outcome) 3
6. Component Design 4
   - 6.1. Validation 4
   - 6.2. Store 4
   - 6.3. HTTP Processing 4
   - 6.4. JSON 4
   - 6.5. Router (Static Dispatch) 4
   - 6.6. Server (Task-Per-Connection) 5
7. Design Patterns 5
   - 7.1. Generics for Code Reuse 5
   - 7.2. Discriminated Records as Result Types 5
   - 7.3. Protected Objects for Concurrency 5
8. Data Flow 5
   - 8.1. Success Path (POST /entries) 5
   - 8.2. Error Path (duplicate key) 6
9. Concurrency Design 6
   - 9.1. Thread Safety 6
   - 9.2. Task Lifecycle 6
10. Performance and Memory Design 6
    - 10.1. Zero-Overhead Abstractions 6
    - 10.2. Memory Management 6
11. Build and Deployment 6
    - 11.1. Build System 6
    - 11.2. Docker Production Image 6
12. Testing Strategy 7
    - 12.1. Test Organization 7
    - 12.2. Test Infrastructure 7
13. Appendices 8
    - 13.1. Change History 8

2. Introduction

2.1. Purpose

This Software Design Specification (SDS) describes the package structure, type definitions, design patterns, concurrency model, and build strategy for **Dictionary_Stdlib**, an educational REST microservice built exclusively with Ada 2022 standard library packages.

2.2. Scope

This document covers:

- flat package hierarchy and dependency flow,
- generic Bounded_Text design and instantiation,
- protected object concurrency model,
- HTTP/1.1 parsing and JSON serialization design,
- static dispatch routing,
- error handling strategy (status enums, discriminated records), and
- build and containerization.

2.3. References

- Software Requirements Specification (SRS) for Dictionary_Stdlib.
- Ada 2022 Language Reference Manual (ISO/IEC 8652:2023).
- GNAT Reference Manual (GNAT.Sockets specification).
- RFC 7231 – HTTP/1.1 Semantics and Content.

3. Architectural Overview

3.1. Architecture Style

This project uses a **flat package structure organized by concern**. It does not use the hybrid DDD/Clean/Hexagonal architecture. That architecture will be used in the Dictionary_Hybrid follow-on project.

The flat structure was chosen because:

- the service is small enough that layered separation adds complexity without proportional benefit,
- the educational focus is on Ada 2022 features and HTTP protocol mechanics, not architecture patterns, and
- a simpler structure is easier for engineers new to Ada to navigate.

3.2. Dependency Flow

All dependencies flow downward. There are no cycles.

```
dictionary_main
  └── Dictionary.Server
        └── Dictionary.Router
              ├── Dictionary.HTTP
              ├── Dictionary.JSON
              │     └── Dictionary.Types
              ├── Dictionary.Store
              │     └── Dictionary.Types
              └── Dictionary.Validation
                    └── Dictionary.Types
                          └── Dictionary.Bounded_Text
```

4. Package Structure

4.1. Directory Layout

All source files live under `src/`. There are no subdirectories.

```
src/
  dictionary.ads
  dictionary_main.adb
  dictionary-bounded_text.ads/adb
  dictionary-http.ads/adb
  dictionary-json.ads/adb
  dictionary-router.ads/adb
  dictionary-server.ads/adb
  dictionary-store.ads/adb
  dictionary-types.ads
  dictionary-validation.ads/adb
```

4.2. Package Descriptions

Package	Purpose
`Dictionary`	Root namespace. Pure, empty. Anchors the child-package hierarchy.
`Dictionary.Bounded_Text`	Generic validated bounded text. Parameterized by max length and character predicate. Applies optional normalization (e.g., lowercase for keys).
`Dictionary.HTTP`	HTTP/1.1 request parsing and response formatting. Defines Request/Response records, method/status enums.
`Dictionary.JSON`	Hand-rolled JSON parse (extract fields by name) and serialize (entry, list, error, health).
`Dictionary.Router`	Static dispatch route matching. Maps method + path to handler functions. Coordinates validation, store, and JSON.
`Dictionary.Server`	TCP listener using GNAT.Sockets. Accept loop spawns a Connection_Task per client.
`Dictionary.Store`	Thread-safe CRUD store. Protected object wrapping Ada.Containers.Ordered_Maps.
`Dictionary.Types`	Domain types: Key_Text/Value_Text instantiations, Entry_Record, Entry_List, Store_Status, Entry_Result.
`Dictionary.Validation`	Character and string predicates with Pre/Post contracts. Defines Is_Key_Char, Is_Value_Char, Is_Valid_Key, Is_Valid_Value.

5. Type Definitions

5.1. Generic Bounded_Text

The central reusable abstraction. One generic, two instantiations.

```
generic
   Max_Length : Positive;
   with function Is_Valid_Char (C : Character) return Boolean;
   with function Normalize (C : Character) return Character is <>;
package Dictionary.Bounded_Text is
   type Text is private;
   function Create (S : String) return Text
```

```
      with Pre => S'Length >= 1
               and then S'Length <= Max_Length
               and then (for all C of S => Is_Valid_Char (C));
   function To_String (T : Text) return String;
   function Length (T : Text) return Natural;
   function "<" (L, R : Text) return Boolean;
   function "=" (L, R : Text) return Boolean;
private
   type Text is record
      Data : String (1 .. Max_Length) := (others => ' ');
      Len  : Natural range 0 .. Max_Length := 0;
   end record;
end Dictionary.Bounded_Text;
```

Design rationale:

- **Why not Ada.Strings.Bounded?** Bounded uses controlled types (heap, finalization), which are not SPARK-compatible. A purpose-built record with a fixed array is simpler, provable, and teaches generic design.
- **Why the Normalize formal?** Keys are case-insensitive. Passing `Ada.Characters.Handling.To_Lower` as Normalize means lowercasing happens once at construction, and all comparisons operate on normalized data.
- **Why the Is_Valid_Char formal?** Different text types have different character sets. The generic parameterizes this without code duplication.

5.2. Instantiations

```
package Key_Text is new Dictionary.Bounded_Text
  (Max_Length => 50, Is_Valid_Char => Is_Key_Char,
   Normalize => Ada.Characters.Handling.To_Lower);

package Value_Text is new Dictionary.Bounded_Text
  (Max_Length => 200, Is_Valid_Char => Is_Value_Char,
   Normalize => Identity);
```

5.3. Entry_Result (Lightweight Option)

```
type Entry_Result (Found : Boolean := False) is record
   case Found is
      when True  => Data : Entry_Record;
      when False => null;
   end case;
end record;
```

A discriminated record acting as an Option type. Returned by `Store.Get` to separate "not found" from "found with data" without exceptions.

5.4. Store_Status (Operation Outcome)

```
type Store_Status is (Success, Already_Exists, Not_Found, Store_Full);
```

Returned as an `out` parameter from write operations. The router maps each status to the appropriate HTTP response code.

6. Component Design

6.1. Validation

Pure functions with Pre/Post contracts. No state, no side effects. Character-level predicates (`Is_Key_Char`, `Is_Value_Char`) are passed as generic formals to Bounded_Text so validation is enforced at construction time.

String-level predicates (`Is_Valid_Key`, `Is_Valid_Value`) are used by the JSON parser and router for input checking before constructing Bounded_Text values.

6.2. Store

A single protected object (`Dictionary_Store`) declared at package level in `Dictionary.Store`.

- **Functions** (concurrent access): `Contains`, `Get`, `Get_All`, `Count`.
- **Procedures** (exclusive access): `Create`, `Update`, `Delete`.
- **Backing structure**: `Ada.Containers.Ordered_Maps` instantiated with `Key_Text.Text` as key and `Value_Text.Text` as element. The `"<"` operator on `Key_Text.Text` provides lexicographic ordering on normalized (lowercase) data, giving sorted iteration for free.
- **Capacity bound**: `Create` checks `Count >= Max_Entries` before insertion.

6.3. HTTP Processing

The HTTP package handles two concerns:

1. **Parsing**: `Parse_Request` scans raw bytes for the request line (`METHOD PATH HTTP/1.1`), extracts headers (specifically `Content-Length`), and copies the body. Uses fixed-size buffers (256 bytes for path, 4096 for body). This is a deliberately simplified HTTP/1.1 subset: the version token is parsed but not validated, and only `Content-Length` framing is supported (no chunked transfer encoding).

2. **Formatting**: `Format_Response` builds a complete HTTP/1.1 response string including status line, `Content-Type`, `Content-Length`, `Connection: close`, and body.

The Request and Response records use fixed arrays for path/body (bounded, stack-allocated) and `Ada.Strings.Unbounded` for the response body (variable-length wire data).

6.4. JSON

Minimal hand-rolled JSON for two payload shapes:

- **Parse**: scan for `"key"` and `"value"` field names, extract quoted string values, handle `\"` and `\\` escapes.
- **Serialize**: string concatenation for entry, list, error, and health responses. Uses `Ada.Strings.Unbounded` internally for building variable-length output.

6.5. Router (Static Dispatch)

`Handle_Request` examines method + path and calls the appropriate handler function directly. No tagged types, no dynamic dispatch.

Route matching order:

1. `/health` – health check
2. `/entries` (exact) – collection operations (GET list, POST create)
3. `/entries/{key}` – single-entry operations (GET, PUT, DELETE)
4. Everything else – 404

Each handler validates input, calls the store, serializes the response, and returns a Response record.

6.6. Server (Task-Per-Connection)

`Dictionary.Server.Start`:

1. Creates a TCP socket with `GNAT.Sockets.Create_Socket`.
2. Sets `SO_REUSEADDR` to allow restart without address-in-use errors.
3. Binds to `Any_Inet_Addr` on the configured port.
4. Enters an infinite `Accept_Socket` loop.

Each accepted connection spawns a dynamically allocated `Connection_Task`:

1. Receives the socket via an entry call (value copy, no dangling reference).
2. Reads raw bytes with `Receive_Socket`, converts to String.
3. Parses the HTTP request.
4. Dispatches through the router.
5. Formats and sends the response with `Send_Socket`.
6. Closes the socket.
7. Task terminates.

Known simplification: dynamically allocated tasks leak memory on termination. A production server would use a bounded task pool.

7. Design Patterns

7.1. Generics for Code Reuse

`Dictionary.Bounded_Text` demonstrates Ada's generic mechanism: one package definition, parameterized by data (max length) and behavior (character predicate, normalization), instantiated twice with different configurations.

7.2. Discriminated Records as Result Types

`Entry_Result` and `Store_Status` replace exceptions for expected outcomes. The router uses `case` statements on these types to map store outcomes to HTTP status codes. This is explicit, exhaustive (compiler-checked), and exception-free at the API boundary.

7.3. Protected Objects for Concurrency

Ada's protected types provide built-in reader/writer locking without external libraries. Functions allow concurrent readers; procedures enforce exclusive access. This is a language feature, not a library pattern.

8. Data Flow

8.1. Success Path (POST /entries)

1. TCP bytes arrive on the socket.
2. `Connection_Task` reads bytes, converts to String.
3. `HTTP.Parse_Request` produces a Request record.
4. `Router.Handle_Request` matches POST + `/entries`.
5. `JSON.Parse_Entry` extracts key and value strings from the body.
6. `Validation.Is_Valid_Key` and `Is_Valid_Value` check the strings.
7. `Key_Text.Create` normalizes the key to lowercase.
8. `Store.Dictionary_Store.Create` inserts the entry (protected, exclusive).
9. `JSON.Serialize_Entry` builds the response body.
10. `HTTP.Format_Response` wraps it in HTTP headers.
11. `Connection_Task` sends bytes, closes socket.

8.2. Error Path (duplicate key)

Steps 1–7 as above. Step 8 returns `Already_Exists`. Step 9 becomes `JSON.Serialize_Error`. The router returns a 409 Conflict response.

9. Concurrency Design

9.1. Thread Safety

The dictionary store is the only shared mutable state. It is protected by an Ada protected object, which guarantees:

- multiple concurrent readers (via protected functions), and
- exclusive writer access (via protected procedures).

No other shared state exists. Each connection task operates on its own socket and local variables.

9.2. Task Lifecycle

Tasks are dynamically allocated and self-terminating. There is no task pool or bounded concurrency limit. This is an intentional simplification for the educational version.

10. Performance and Memory Design

10.1. Zero-Overhead Abstractions

- Static dispatch routing (no vtable lookup).
- Generic instantiation resolved at compile time (no runtime polymorphism).
- Bounded_Text uses stack-allocated fixed arrays (no heap for domain data).

10.2. Memory Management

- Domain types (`Key_Text.Text`, `Value_Text.Text`, `Entry_Record`) are stack-allocated with fixed maximum sizes.
- HTTP response building uses `Ada.Strings.Unbounded` (heap) in the I/O layer only.
- The Ordered_Maps container uses heap allocation internally (standard Ada container behavior).
- Connection tasks are heap-allocated and leak on termination (documented simplification).

11. Build and Deployment

11.1. Build System

- **Alire** manages the toolchain (GNAT FSF 15.x, GPRbuild 25.x).
- **GPRbuild** compiles the project via `dictionary_stdlib.gpr`.
- **Make** orchestrates build, test, and Docker targets.
- Strict compiler switches: `-gnatwa` (all warnings), `-gnatVa` (validity checks), full style checks.

11.2. Docker Production Image

Multi-stage build:

1. **Builder stage**: `ghcr.io/abitofhelp/dev-container-ada:latest` compiles the release binary.
2. **Runtime stage**: `ubuntu:22.04` with only curl and the binary. Runs as nonroot user `app` (UID 10000). Read-only root filesystem. Health check on `/health`.

Image size: 93 MB.

12. Testing Strategy

12.1. Test Organization

Suite	Count	Scope
Unit	99	Validation, Bounded_Text, JSON, HTTP parsing, Store operations
Integration	26	Router pipeline: request record in, response record out
E2E	28	curl against a running server, all endpoints and error paths

12.2. Test Infrastructure

- Custom `Test_Framework` package (assert, report, pass/fail counts, exit code).
- Separate GPR per suite (`test/unit/unit_tests.gpr`, `test/integration/integration_tests.gpr`).
- E2E uses a shell script (`test/e2e/e2e_test.sh`) that starts the server, runs curl, and verifies status codes.

13. Appendices

13.1. Change History

Version	Date	Author	Changes
0.1.0	2026-04-07	Michael Gardner	Initial SDS for the dictionary_stdlib educational microservice.